

# 권한을 안 줬는데 데이터가 샀다

접근 제어 모델의 진화

# 어떻게 가능했을까?

---

A에게만 열람 권한을 준 기밀 파일을  
권한 없는 B가 읽어냈다.

관리자는 분명 A한테만 GRANT 했다

# 트로이 목마 시나리오

포비

우테코 지원자 평가 X 파일  
소유자 (Own)

네오

X 파일(평가 파일) (R),  
파일 Y (W)

새로이 (공격자)

파일 X 권한 없음  
파일 Y (R)

# 공격 과정

---

- ① 새로이가 LMS+(트로이 목마)을 만든다
- ② 네오에게 건넨다 — '이전 LMS가 별로여서요. 이거 한번 써보세요'
- ③ 네오가 실행 → 프로그램이 네오의 권한으로 동작
- ④ 파일 X를 읽어서(R) → 파일 Y에 복사(W)
- ⑤ 새로이가 파일 Y를 읽는다 → 기밀 유출 완료

# 원인: *DAC*의 구조적 한계

---

## 프로그램이 사용자 권한을 그대로 물려받는다

신분 도용

— ID 탈취 시 모든 권한 탈취

트로이 목마

— 프로그램이 사용자 권한 행사

중앙 통제 불가

— 소유자 개인이 권한 관리

DAC = Discretionary Access Control (임의적 접근 제어)

# 그럼 어떻게 막을 수 있을까?

---

소유자가 아닌, 시스템이 강제하는 보안

# *MAC* — 강제적 접근 제어

---

## **No Read Up**

— 자기 등급 이상은 못 읽는다

## **No Write Down**

— 자기 등급 이하에 못 쓴다

**No Write Down 덕분에  
트로이 목마의 유출 경로가 원천 차단**

# *BLP vs Biba* — 쌍둥이 모델

|       | BLP           | Biba         |
|-------|---------------|--------------|
| 목적    | 기밀성 보호        | 무결성 보호       |
| 읽기    | No Read Up    | No Read Down |
| 쓰기    | No Write Down | No Write Up  |
| 사용 분야 | 군, 국방         | 금융           |
| 한계    | 무결성 미보장       | 기밀성 미보장      |

# MAC은 너무 뻑세다

---

- 모든 주체·객체에 보안 등급을 일일이 매겨야 한다
- 사용자 기능이 극도로 제한된다
- 군사·국방에는 맞지만, 일반 회사에서는 쓸 수 없다

실무에서는 뭘 쓸까? →

# *RBAC* — 역할 기반 접근 제어

---

사용자 — [UA] → 역할 — [PA] → 권한

개인별 권한 관리 → 역할 단위로 묶어서 관리

퇴사자 권한 회수 누락 → 역할만 회수하면 끝

너무 엄격하거나 느슨 → 조직 구조에 맞는 유연한 설계

# 임무 분리(*SoD*) & 역할 폭발

---

## *SoD*

출납(Cashier) + 청구(Billing) = 횡령 가능

→ SSD: 두 역할 동시 배정 금지

→ DSD: 동시 활성화 금지

---

## 역할 폭발

조직이 커지면 역할 수가 기하급수적 증가 → 관리 불가

# ABAC — 동적 환경의 답

주체 속성 + 객체 속성 + 환경 조건

병원 시나리오

[내과 소속] AND [환자 A 담당] 의사가  
[오전 9시~오후 6시]에 [A동 10호]에 있을 때만

→ 환자 A의 의료 정보 열람 허용

실무: RBAC + ABAC 혼합 사용이 트렌드

# 접근 제어 모델의 진화

***DAC***

(자유로움, 취약)

→ 트로이 목마에 뚫린다



***MAC***

(강력, 경직)

→ 실무에서 못 쓴다



***RBAC***

(현실적 절충)

→ 역할이 폭발한다



***ABAC***

(동적, 세밀)

완벽한 하나의 모델은 없다 — 상황에 맞는 선택과 조합이 핵심